

Chương này trình bày quá trình phân tích thiết kế, hiện thực và đánh giá hệ thống đặt vé và quản lý sự kiện được phát triển trong đề án. Trước hết, chương làm rõ kiến trúc phần mềm được lựa chọn và cách áp dụng kiến trúc đó vào hệ thống thực tế. Tiếp theo, chương mô tả thiết kế tổng quan, thiết kế chi tiết, quá trình xây dựng ứng dụng, kết quả đạt được, kiểm thử và định hướng triển khai. Các nội dung trong chương này đóng vai trò kết nối giữa phần khảo sát yêu cầu ở các chương trước với phần đóng góp và tổng kết ở các chương sau, qua đó cho thấy hệ thống đã được tổ chức và hiện thực hóa như thế nào trên cả phía ứng dụng di động lẫn phía máy chủ.

0.1 Thiết kế kiến trúc

0.1.1 Lựa chọn kiến trúc phần mềm

Đối với bài toán đặt vé và quản lý sự kiện trên thiết bị di động, hệ thống cần đồng thời hỗ trợ nhiều chức năng khác nhau như quản lý tài khoản, tìm kiếm sự kiện, quản lý sự kiện, đặt vé, thanh toán, nhắn tin, thông báo và check-in bằng mã mã phản hồi nhanh (Quick Response - QR). Các chức năng này có liên hệ chặt chẽ với nhau, dùng chung dữ liệu và cùng phục vụ một luồng nghiệp vụ xuyên suốt từ lúc người dùng khám phá sự kiện cho tới lúc tham dự thực tế. Vì vậy, kiến trúc phù hợp nhất cho hệ thống trong phạm vi đề án là kiến trúc client-server theo hướng mobile-first, kết hợp với tổ chức phần mềm theo mô hình ba lớp ở phía máy chủ. Theo cách hiểu này, ứng dụng di động đóng vai trò lớp giao diện và tương tác với người dùng, máy chủ backend đóng vai trò lớp xử lý nghiệp vụ, còn cơ sở dữ liệu là lớp lưu trữ dữ liệu tập trung.

Kiến trúc ba lớp được lựa chọn thay cho các hướng như microservice hay backendless vì nó phù hợp hơn với quy mô hiện tại của hệ thống. Nếu sử dụng microservice, mỗi nhóm chức năng như tài khoản, sự kiện, vé, thông báo hay chat có thể được tách thành các dịch vụ độc lập. Tuy nhiên, với phạm vi của đề án, cách tiếp cận đó làm tăng độ phức tạp triển khai, quản lý giao tiếp giữa các dịch vụ và đồng bộ dữ liệu, trong khi lượng người dùng và số lượng giao dịch chưa ở mức cần phân tán mạnh. Ngược lại, kiến trúc đơn khối có tổ chức theo lớp giúp hệ thống dễ xây dựng, dễ bảo trì hơn trong giai đoạn phát triển đầu tiên, đồng thời vẫn đủ rõ ràng để mở rộng khi cần. So với mô hình backendless, việc xây dựng backend riêng còn cho phép kiểm soát chặt chẽ hơn các nghiệp vụ đặc thù như quản lý nhiều hạng vé, kiểm soát trạng thái thanh toán, gửi lời mời, kiểm tra vé đã dùng hay chưa và xác nhận check-in theo sự kiện cụ thể.

Xét trên hệ thống cụ thể của đề tài, lớp giao diện và tương tác được hiện thực bằng ứng dụng di động React Native kết hợp Expo. Thành phần này bao gồm các

màn hình chức năng trong thư mục `frontend/pages`, các thành phần giao diện dùng lại trong `frontend/components`, hệ điều hướng trong `frontend/-navigators`, cùng các ngữ cảnh quản lý trạng thái như xác thực và bản địa hóa trong `frontend/context`. Nhiệm vụ của lớp này là tiếp nhận thao tác của người dùng, tổ chức luồng di chuyển giữa các màn hình, hiển thị dữ liệu và gửi yêu cầu đến backend thông qua các service như `auth.service`, `event.service`, `ticket.service`, `notification.service` hay `chat.service`. Nói cách khác, đây là phần gần nhất với người dùng cuối và chịu trách nhiệm thể hiện toàn bộ chức năng của hệ thống trên thiết bị di động.

Lớp xử lý nghiệp vụ được tổ chức ở phía backend theo một dạng MVC điều chỉnh cho ứng dụng API và dịch vụ thời gian thực. Trong hệ thống này, vai trò tương đương với phần Model được thể hiện bởi các schema và model Mongoose trong thư mục `backend/src/models`, nơi quản lý cấu trúc dữ liệu của người dùng, sự kiện, vé, thanh toán, thông báo, bookmark, chatroom và message. Vai trò tương đương với phần Controller là các controller trong `backend/src/controllers`, phụ trách xử lý từng luồng nghiệp vụ như đăng nhập, tạo sự kiện, đặt vé, xác nhận thanh toán, gửi lời mời, nhắn tin hay check-in. Bổ sung cho đó là các middleware và service trong `backend/src/middleware`, `backend/src/services` và `backend/src/utisls`, dùng để giải quyết những nghiệp vụ dùng chung như xác thực JWT, gửi email, gửi thông báo đẩy, sinh mã QR, gọi ý nội dung sự kiện bằng AI hoặc lập lịch nhắc sự kiện. Phần View trong mô hình MVC truyền thống không được đặt ở phía backend như các ứng dụng web server-rendered, mà được chuyển sang ứng dụng di động ở frontend. Đây là điểm điều chỉnh quan trọng để phù hợp với kiến trúc ứng dụng di động hiện đại.

Ở mức tổ chức truy cập, các route trong `backend/src/routes` đóng vai trò cổng vào của lớp dịch vụ. Mỗi nhóm route như `/api/auth`, `/api/users`, `/api/events`, `/api/tickets`, `/api/notifications` hay `/api/chat` tiếp nhận yêu cầu từ ứng dụng khách, sau đó chuyển tiếp cho controller tương ứng. Điều này tạo nên một chuỗi xử lý tương đối rõ ràng: người dùng thao tác trên giao diện di động, service ở frontend gửi yêu cầu HTTP hoặc sự kiện socket tới backend, route tiếp nhận yêu cầu, controller xử lý nghiệp vụ, model truy cập dữ liệu, sau đó kết quả được trả ngược về giao diện. Với các chức năng gần thời gian thực như chat, hệ thống mở rộng thêm một kênh giao tiếp bằng Socket.IO ở tệp `backend/src/socket.ts`. Đây có thể xem là một thành phần bổ sung cho lớp xử lý nghiệp vụ, giúp hệ thống vượt ra ngoài mô hình request-response thuần túy mà vẫn không phá vỡ cấu trúc tổng thể.

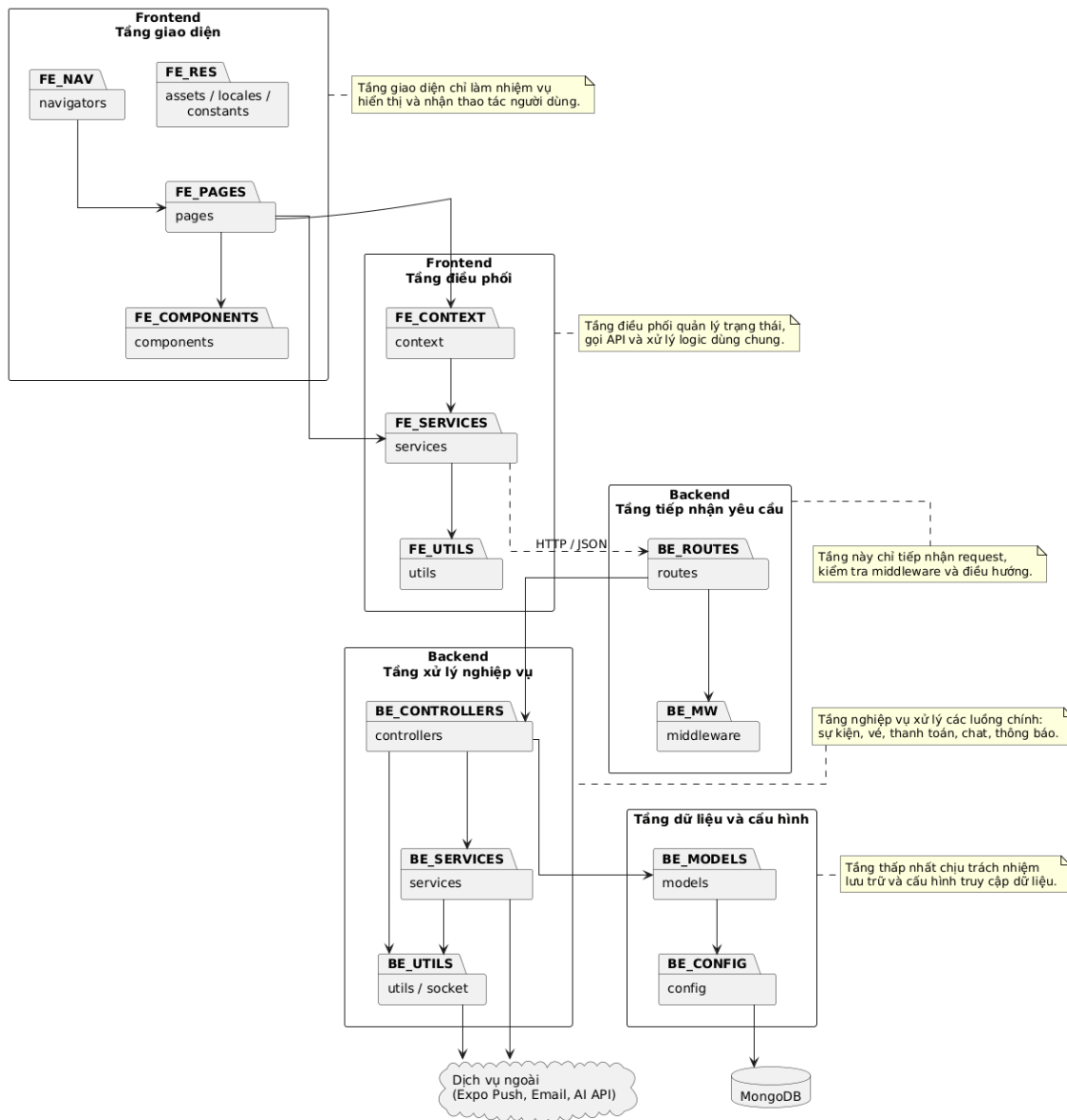
Lớp dữ liệu của hệ thống được xây dựng trên MongoDB và được truy cập thông

qua Mongoose. Việc dùng cơ sở dữ liệu tài liệu giúp hệ thống linh hoạt trong việc biểu diễn các thực thể có cấu trúc thay đổi theo nghiệp vụ, chẳng hạn sự kiện có thể có nhiều hạng vé, trạng thái duyệt, thông tin tọa độ hoặc dữ liệu liên quan đến người tổ chức. Với bài toán của đề tài, dữ liệu không chỉ cần được lưu trữ mà còn phải phản ánh quan hệ giữa nhiều thực thể như người dùng – sự kiện – vé – thanh toán – thông báo. Bởi vậy, lớp dữ liệu không chỉ đóng vai trò lưu trữ mà còn là nền tảng để bảo đảm tính nhất quán cho các thao tác nghiệp vụ quan trọng như tăng số lượng vé đã bán, đánh dấu vé đã check-in hoặc ghi nhận lời mời đã gửi.

Từ góc nhìn tổng thể, kiến trúc được áp dụng cho hệ thống có thể mô tả là kiến trúc client-server ba lớp, trong đó backend được tổ chức theo tinh thần MVC điều chỉnh cho ứng dụng API. Đây là lựa chọn phù hợp với sản phẩm của đề án vì vừa bảo đảm tính mạch lạc trong thiết kế, vừa đủ linh hoạt để tích hợp thêm các thành phần mở rộng như thông báo đẩy, chat thời gian thực, AI hỗ trợ nội dung và kiểm soát tham dự bằng mã QR. Quan trọng hơn, kiến trúc này cho phép nhóm chức năng của hệ thống được phân chia tương đối rõ ràng, giúp cho việc phát triển, bảo trì và mở rộng về sau thuận lợi hơn so với một cấu trúc mã nguồn rời rạc hoặc quá phức tạp so với quy mô hiện tại của đề tài.

0.1.2 Thiết kế tổng quan

Biểu đồ gói tổng quan của hệ thống được xây dựng theo hướng phân tầng rõ ràng giữa phía ứng dụng di động và phía máy chủ. Mục tiêu của cách tổ chức này là giúp các thành phần đảm nhiệm đúng trách nhiệm của mình, hạn chế sự phụ thuộc chéo và thuận lợi cho việc bảo trì sau này. Ở mức khái quát, hệ thống được chia thành các nhóm gói chính gồm: nhóm giao diện người dùng ở frontend, nhóm điều phối và truy cập dịch vụ ở frontend, nhóm tiếp nhận yêu cầu ở backend, nhóm xử lý nghiệp vụ và tích hợp ở backend, cùng với nhóm dữ liệu và cấu hình ở tầng thấp nhất.



Hình 0.1: Biểu đồ gói tổng quan của hệ thống event-booking-app

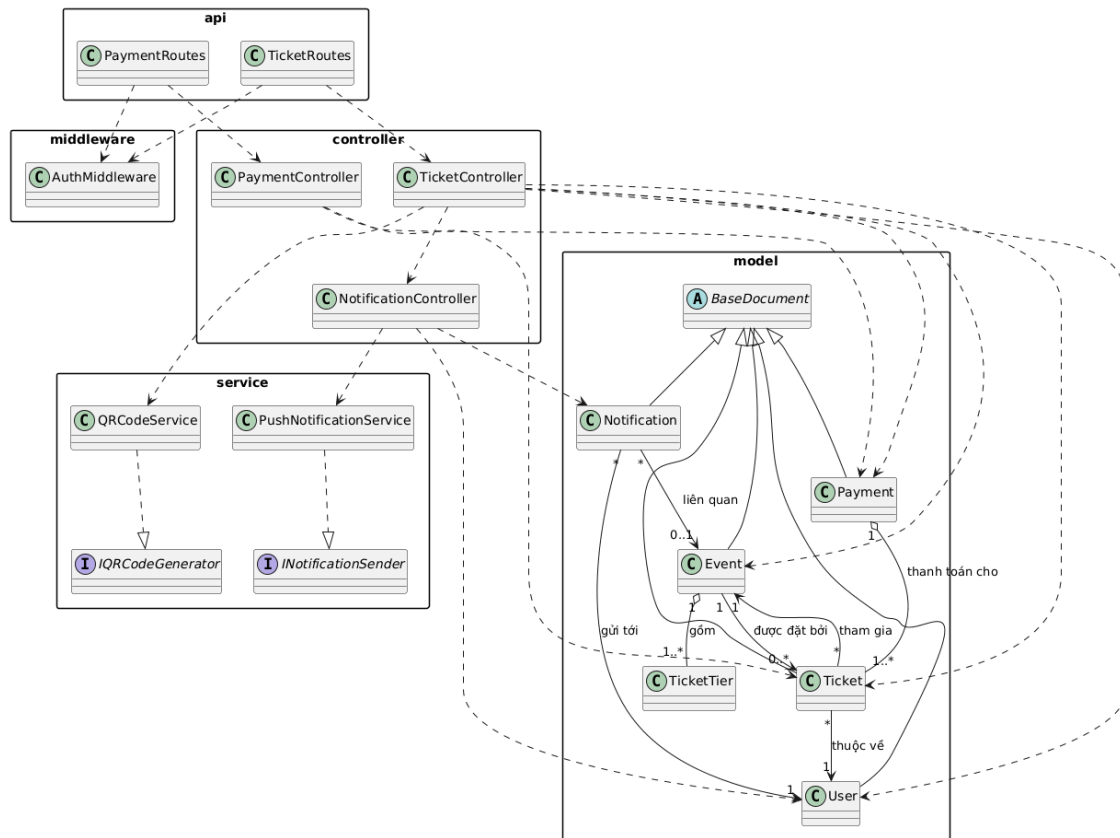
Ở phía frontend, các package có thể được chia thành hai tầng chính. Tầng giao diện bao gồm pages, components, navigators, assets, locales và constants. Trong đó, pages chứa các màn hình nghiệp vụ cụ thể như đăng ký, đặt vé, quản lý sự kiện, chat và thông báo; components chứa các thành phần giao diện dùng lại; navigators chịu trách nhiệm tổ chức luồng điều hướng; còn constants, assets và locales cung cấp các tài nguyên dùng chung. Tầng điều phối ở frontend bao gồm context, services và utils. Package context lưu trạng thái dùng chung như thông tin xác thực và ngôn ngữ, package services thực hiện giao tiếp với backend thông qua API, còn utils xử lý các hàm phụ trợ. Cấu trúc này đảm bảo giao diện không truy cập trực tiếp tới backend hay dữ liệu lưu trữ, mà luôn đi qua tầng service và context tương ứng.

Ở phía backend, package routes là lớp tiếp nhận yêu cầu từ bên ngoài và phân

phối yêu cầu đến các controller tương ứng. Package `middleware` cung cấp các cơ chế kiểm tra xác thực, phân quyền và xử lý lỗi dùng chung cho toàn hệ thống. Package `controllers` là trung tâm điều phối nghiệp vụ, nơi hiện thực các luồng như đăng nhập, tạo sự kiện, đặt vé, gửi lời mời, check-in và xử lý thông báo. Package `services` và `utils` bổ sung các chức năng hỗ trợ hoặc tích hợp như sinh gợi ý nội dung sự kiện bằng AI, gửi email, gửi thông báo đẩy, lập lịch nhắc sự kiện, sinh mã QR và xử lý socket thời gian thực. Package `models` đại diện cho tầng dữ liệu nghiệp vụ, mô tả các thực thể như người dùng, sự kiện, vé, thanh toán, thông báo, bookmark, chatroom và tin nhắn. Cuối cùng, package `config` chịu trách nhiệm kết nối cơ sở dữ liệu và nạp biến môi trường, tạo nền tảng kỹ thuật cho toàn bộ các package phía trên.

0.1.3 Thiết kế chi tiết gói

Để làm rõ cách tổ chức các lớp trong hệ thống, đề tài lựa chọn nhóm package giải quyết nghiệp vụ đặt vé, thanh toán và tham dự sự kiện để mô tả chi tiết. Đây là nhóm package quan trọng vì nó kết nối trực tiếp nhiều thực thể nghiệp vụ nhất của hệ thống, đồng thời bao trùm các thao tác cốt lõi như chọn vé, xử lý thanh toán, sinh vé điện tử, gửi thông báo và check-in bằng mã QR. Ở mức thiết kế gói, biểu đồ tập trung vào năm nhóm thành phần chính là `api`, `middleware`, `controller`, `service` và `model`. Mỗi nhóm được phân công một vai trò rõ ràng nhằm tránh chồng chéo trách nhiệm và giúp luồng xử lý nghiệp vụ có thể theo dõi được từ đầu tới cuối.



Hình 0.2: Biểu đồ thiết kế gói cho nghiệp vụ đặt vé, thanh toán và tham dự sự kiện

Trong thiết kế này, package `api` gồm các lớp như `TicketRoutes` và `PaymentRoutes`, chịu trách nhiệm tiếp nhận yêu cầu từ phía ứng dụng khách và chuyển yêu cầu sang tầng điều phối. Package `middleware` chứa lớp `AuthMiddleware`, được dùng để kiểm tra token xác thực và bảo vệ các chức năng chỉ dành cho người dùng đã đăng nhập. Package `controller` bao gồm các lớp như `TicketController`, `PaymentController` và `NotificationController`. Đây là nơi tổ chức luồng nghiệp vụ chính, chẳng hạn kiểm tra loại vé, khởi tạo giao dịch, xác nhận thanh toán, gọi dịch vụ sinh mã QR, cập nhật trạng thái vé và gửi thông báo sau giao dịch. Package `service` bao gồm các dịch vụ hỗ trợ như `QRCodeService` và `PushNotificationService`, cùng với các interface tương ứng như `IQRCodeGenerator` và `INotificationSender`. Việc đưa interface vào thiết kế giúp tách biệt giữa phần hợp đồng chức năng và phần hiện thực cụ thể, thuận lợi cho việc thay thế hoặc mở rộng dịch vụ về sau. Package `model` bao gồm các lớp dữ liệu chính như `Event`, `Ticket`, `Payment`, `Notification`, `User` và `TicketTier`, cùng lớp cơ sở trừu tượng `BaseDocument`.

Các quan hệ giữa các lớp trong biểu đồ gói thể hiện rõ cách vận hành của hệ thống. Quan hệ phụ thuộc (dependency) xuất hiện khi các lớp route sử dụng mid-

dleware và controller, hoặc khi controller cần gọi service và truy cập model để xử lý nghiệp vụ. Quan hệ thực thi (implementation) xuất hiện ở chỗ `QRCodeService` hiện thực `IQRCodeGenerator` và `PushNotificationService` hiện thực `INotificationSender`. Quan hệ kế thừa (inheritance) được dùng để mô tả việc các model nghiệp vụ kế thừa từ lớp cơ sở `BaseDocument`. Quan hệ kết tập (aggregation) có thể thấy trong trường hợp `Payment` tập hợp nhiều đối tượng `Ticket` thuộc cùng một giao dịch, trong khi quan hệ hợp thành (composition) thể hiện ở việc `Event` bao gồm nhiều `TicketTier` như một phần cấu thành của sự kiện. Ngoài ra, quan hệ kết hợp (association) được dùng để phản ánh các mối liên hệ nghiệp vụ như mỗi `Ticket` gắn với một `User` và một `Event`, hoặc một `Notification` được gửi tới một `User` và có thể liên quan tới một `Event` cụ thể.

0.2 Thiết kế chi tiết

0.2.1 Thiết kế giao diện

Hệ thống được định hướng là một ứng dụng mobile-first, do đó thiết kế giao diện tập trung chủ yếu vào môi trường điện thoại thông minh và các thiết bị di động cầm tay. Về mặt mục tiêu hiển thị, ứng dụng hướng tới các màn hình có độ phân giải phổ biến trên smartphone hiện nay, hỗ trợ tốt ở cả các thiết bị cỡ nhỏ, thiết bị kích thước trung bình và máy tính bảng. Trong mã nguồn hiện tại, nhiều màn hình như trang chủ, trang chi tiết sự kiện, màn hình chọn vé hay màn hình chat đều sử dụng cơ chế `useWindowDimensions` để điều chỉnh giao diện theo chiều rộng thực tế của thiết bị. Điều này cho thấy thiết kế giao diện của hệ thống không cố định ở một kích thước duy nhất, mà hướng tới khả năng thích nghi với nhiều khung hiển thị. Trên thiết bị nhỏ, khoảng cách lề, chiều cao ô nhập liệu, kích thước biểu tượng và cỡ chữ được thu gọn để bảo đảm không tràn bố cục; trong khi với máy tính bảng, các thành phần được mở rộng hợp lý để tận dụng không gian hiển thị.

Về số lượng màu sắc và định hướng hiển thị, hệ thống sử dụng môi trường giao diện đồ họa tiêu chuẩn của thiết bị di động với dải màu đầy đủ, hỗ trợ hiển thị văn bản, ảnh, icon, nền khối, gradient và phản hồi trạng thái bằng màu. Trong mã nguồn, bảng màu của ứng dụng được chuẩn hóa tập trung trong các tệp `frontend/constants/colors.ts` và `frontend/constants/theme.ts`. Màu chủ đạo của hệ thống là cam `#F76B10`, được dùng để nhấn mạnh các thao tác chính như đặt vé, xác nhận hoặc điều hướng tới các chức năng nổi bật. Bên cạnh đó, hệ thống sử dụng thêm các màu phụ như xanh dương, vàng, xanh lá, trắng, xám và các màu trạng thái để biểu diễn thông tin khác nhau. Cách tổ chức này giúp giao diện có tính thống nhất cao và hỗ trợ việc thay đổi chủ đề hoặc mở rộng sang dark mode về sau.

Nguyên tắc tổ chức giao diện

Thiết kế giao diện của hệ thống được xây dựng theo hướng ưu tiên khả năng thao tác nhanh, dễ hiểu và phù hợp với hành vi sử dụng trên điện thoại. Các màn hình nghiệp vụ chính đều tuân theo cấu trúc ba phần: vùng điều hướng phía trên, vùng nội dung trung tâm và vùng thao tác chính ở cuối màn hình hoặc tại những vị trí dễ chạm. Ví dụ, trang chủ gồm phần tiêu đề tài khoản và vị trí người dùng, thanh tìm kiếm, các nhóm danh mục và danh sách sự kiện; màn hình chi tiết sự kiện tổ chức nội dung theo trình tự banner – thông tin sự kiện – thao tác chính; còn màn hình mua vé và thanh toán được thiết kế theo luồng nhập liệu từng bước. Việc phân chia như vậy giúp người dùng nhanh chóng nhận biết phần nào dùng để xem thông tin, phần nào dùng để thao tác và phần nào là phản hồi của hệ thống.

Các thành phần giao diện cũng được chuẩn hóa theo một số quy tắc nhất quán. Nút thao tác chính thường dùng nền đậm hoặc màu nhấn, bo góc lớn và chữ đậm để tạo sự nổi bật. Các ô nhập liệu sử dụng nền sáng, viền rõ ràng, khoảng đệm tương đối lớn để dễ nhập trên màn hình cảm ứng. Các danh sách, thẻ sự kiện và vùng thông tin dùng bo góc mềm để giữ tính nhất quán với tổng thể thiết kế hiện đại của ứng dụng. Với các chức năng có tính hệ quả trực tiếp như thanh toán, xóa hoặc từ chối sự kiện, giao diện luôn cố gắng cung cấp dấu hiệu nhận biết rõ ràng thông qua màu sắc, vị trí nút và thông báo phản hồi. Hệ thống cũng sử dụng icon phổ biến như menu, tìm kiếm, vị trí, tin nhắn, vé và các trạng thái tăng giảm số lượng để giảm khối lượng văn bản phải hiển thị trên màn hình.

Chuẩn hóa vị trí thông điệp và phản hồi hệ thống

Một yêu cầu quan trọng của ứng dụng di động là thông điệp phản hồi phải xuất hiện đúng lúc và đủ rõ để người dùng không bị mất phương hướng khi thao tác. Trong thiết kế hiện tại, thông điệp phản hồi được tổ chức theo ba mức. Mức thứ nhất là phản hồi cục bộ ngay trong thành phần giao diện, ví dụ trạng thái nút bị mờ khi không thể bấm, số lượng vé không thể tăng vượt quota hoặc biểu tượng thay đổi theo trạng thái đã lưu. Mức thứ hai là phản hồi qua các hộp thoại cảnh báo hoặc thông báo ngắn khi thao tác thành công hay thất bại, chẳng hạn khi thêm thẻ, duyệt sự kiện hoặc gặp lỗi kết nối. Mức thứ ba là phản hồi ở mức hệ thống thông qua notification và push notification, áp dụng cho các tình huống như lời mời sự kiện, nhắc lịch, thanh toán thành công hoặc có tin nhắn mới. Việc phân tách phản hồi như vậy giúp cho người dùng luôn nhận được thông tin phù hợp với mức độ quan trọng của thao tác.

Định hướng thiết kế cho các màn hình quan trọng

Đối với màn hình trang chủ, định hướng thiết kế là ưu tiên khám phá nhanh. Người dùng cần nhìn thấy ngay thông tin cá nhân tối thiểu, thanh tìm kiếm, danh mục sự kiện và các sự kiện nổi bật. Vì vậy, bố cục được tổ chức theo chiều dọc, dùng thẻ sự kiện và các thanh danh mục nằm ngang để giảm thao tác cuộn không cần thiết. Đối với màn hình chi tiết sự kiện, thiết kế hướng tới việc cung cấp thông tin đầy đủ nhưng không gây quá tải, nên banner hình ảnh được đặt ở đầu trang, sau đó là khối thông tin tóm tắt, mô tả, thông tin người tổ chức và các hành động như đặt vé, nhắn tin hoặc lưu sự kiện. Đối với màn hình mua vé, thiết kế cần làm nổi bật việc lựa chọn hạng vé, số lượng và tổng tiền để người dùng dễ ra quyết định. Còn đối với màn hình chat, thiết kế được tổ chức theo mẫu hội thoại quen thuộc với khung tin nhắn, thời gian gửi, ô nhập liệu và phản hồi đọc tin nhằm giảm thời gian làm quen của người sử dụng.

Nhận xét chung về thiết kế giao diện

Thiết kế giao diện của hệ thống được xây dựng theo hướng thực dụng, ưu tiên tính dễ dùng và sự nhất quán hơn là những hiệu ứng trình diễn phức tạp. Điểm mạnh của thiết kế nằm ở việc chuẩn hóa tương đối rõ ràng màu, kiểu chữ, vùng nhập liệu, biểu tượng và phản hồi hệ thống, đồng thời có xem xét tới khả năng thích nghi với nhiều kích thước màn hình khác nhau. Dù đây mới là một sản phẩm ở mức thử nghiệm, định hướng giao diện hiện tại đã đủ để làm nền tảng cho các luồng nghiệp vụ chính như khám phá sự kiện, đặt vé, thanh toán, quản lý vé và tương tác giữa các bên. Trong các bước tiếp theo, phần minh họa bằng ảnh giao diện thực tế có thể tiếp tục được bổ sung để làm rõ hơn cách những nguyên tắc thiết kế này được hiện thực hóa trên từng màn hình cụ thể.

Hình ảnh Wireframe 1 số giao diện:

Tạo sự kiện

Ảnh đại diện

Chon ảnh

Tiêu đề sự kiện

Workshop Mobile Event App

Danh mục

Music

Ngày tổ chức

2026-07-15

Thời gian

19:30

Địa điểm

Hà Nội

Mô tả

Mô tả ngắn về sự kiện...

Hạng vé

Hạng vé	Giá vé	Quota	
Standard	100000	100	
VIP	250000	30	

+ Thêm hạng vé

Gợi ý nội dung bằng AI

Lưu nháp

Gửi duyệt

Hình 0.3: Wireframe minh họa giao diện chức năng tạo sự kiện

Đặt vé

Sự kiện

Mobile Event Booking Workshop

Địa điểm

Hà Nội

Thời gian

19:30 - 15/07/2026

Loại vé	Giá	Còn lại	Chọn
Standard	100000	100	
VIP	250000	30	

Số lượng

- 2 +

Thành tiền

200000 VNĐ

Phương thức thanh toán

Wallet

Mã giảm giá

Xác nhận thanh toán

Hình 0.4: Wireframe minh họa giao diện chức năng đặt vé

0.2.2 Thiết kế lớp

Trong hệ thống đặt vé và quản lý sự kiện của đồ án, có nhiều lớp dữ liệu và lớp xử lý cùng phối hợp để thực hiện các nghiệp vụ. Tuy nhiên, nếu xét theo mức độ trung tâm đối với luồng nghiệp vụ chính, bốn lớp quan trọng nhất là `User`, `Event`, `Ticket` và `Payment`. Đây là các lớp phản ánh trực tiếp ba thực thể cốt lõi của hệ thống là người dùng, sự kiện và giao dịch đặt vé. Những lớp khác như `Notification`, `Bookmark`, `ChatRoom`, `Message` hay các lớp tiện ích hỗ trợ sẽ được xem là các lớp bổ trợ.

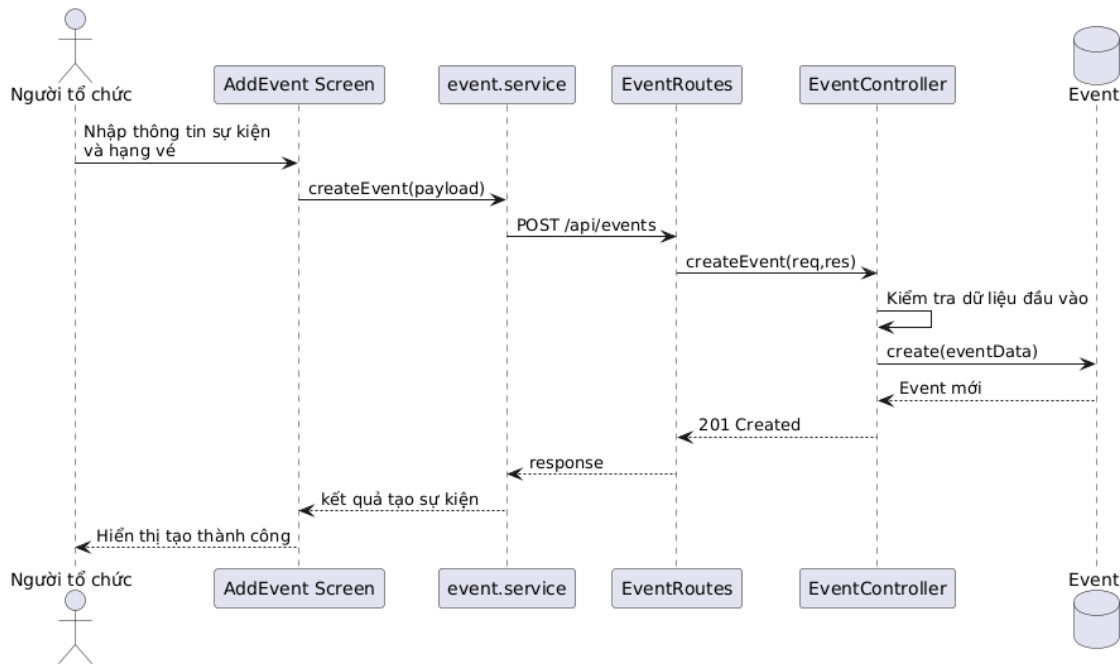
Lớp User

Lớp `User` đại diện cho người sử dụng hệ thống, bao gồm cả người dùng thông thường lẫn quản trị viên. Về mặt thuộc tính, lớp này lưu trữ các thông tin định danh và hồ sơ như `name`, `email`, `password`, `avatar`, `country`, `interests`, `location`, `description`, cùng với các thông tin điều khiển nghiệp vụ như `role`, `verified`, `notificationsEnabled`, `expoPushToken`, `passwordResetCode` và `passwordResetExpires`. Các thuộc tính này cho phép hệ thống không chỉ xác thực người dùng mà còn cá nhân hóa hồ sơ, quản lý quyền truy cập và hỗ trợ các chức năng thông báo đẩy hoặc khôi phục mật khẩu.

Về phương thức, lớp `User` có phương thức quan trọng `matchPassword()` để đối chiếu mật khẩu người dùng nhập vào với mật khẩu đã mã hóa trong cơ sở dữ liệu. Ngoài ra, lớp còn tham gia vào cơ chế tiền xử lý trước khi lưu thông qua thao tác mã hóa mật khẩu bằng `pre-save hook`. Về ý nghĩa thiết kế, `User` là lớp đầu mối cho hầu hết các use case của hệ thống vì mọi nghiệp vụ như đặt vé, tạo sự kiện, nhận thông báo hay nhấn tin đều gắn với một người dùng cụ thể.

Lớp Event

Lớp `Event` đại diện cho một sự kiện được công bố trên hệ thống. Các thuộc tính chính của lớp bao gồm `title`, `description`, `category`, `price`, `date`, `time`, `location`, `coordinates`, `images`, `member`, `attendees`, `rating`, `status`, `approvalStatus`, `organizer` và `ticketTiers`. Trong đó, `organizer` liên kết sự kiện với người tạo sự kiện, còn `ticketTiers` cho phép một sự kiện có nhiều hạng vé khác nhau, mỗi hạng có tên, giá, quota và số lượng đã bán.

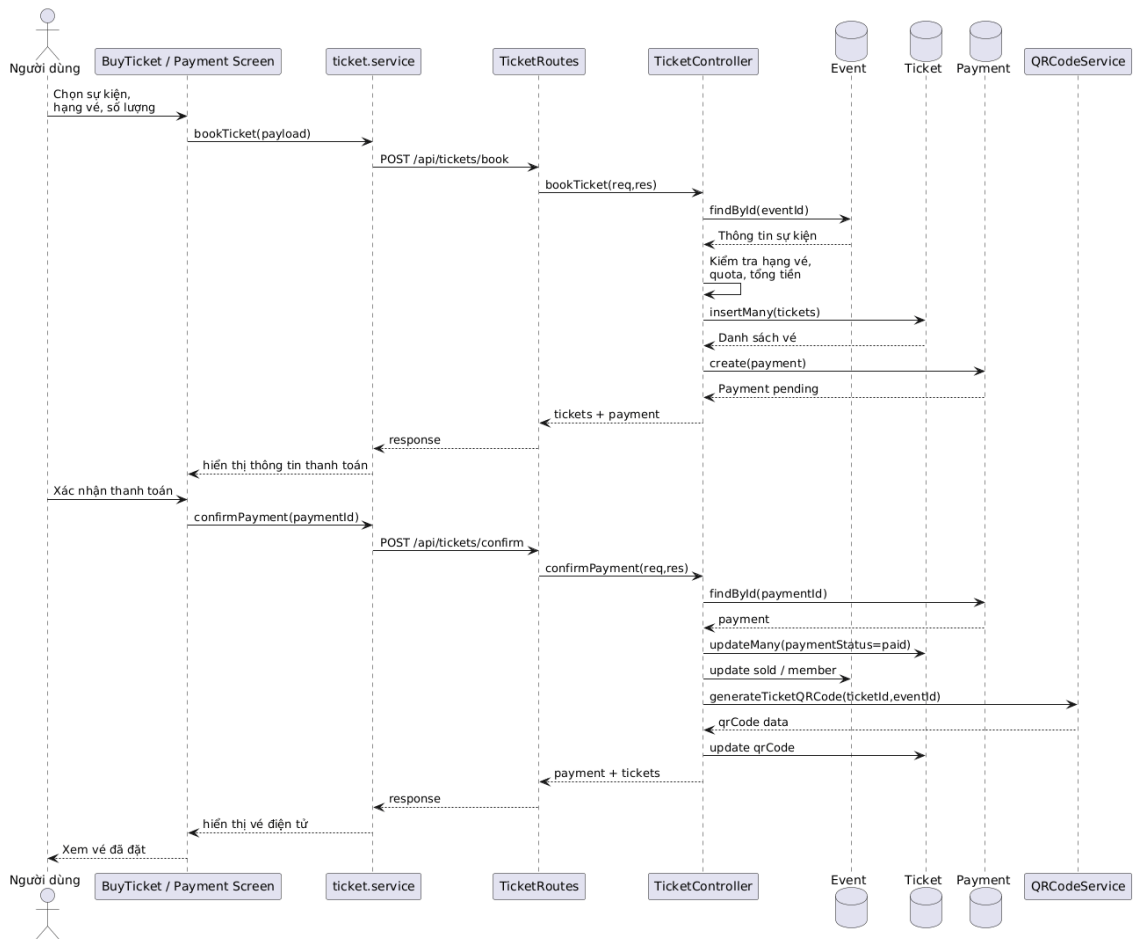


Hình 0.5: Biểu đồ trình tự cho use case Tạo sự kiện

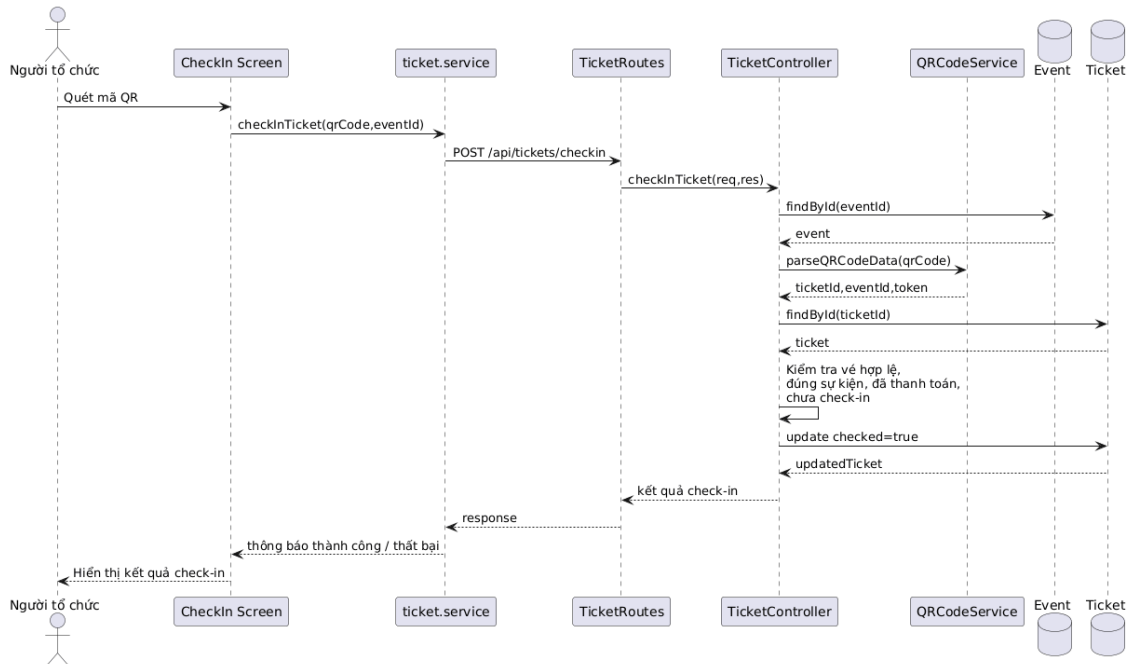
Về mặt phương thức, lớp `Event` không biểu diễn các phương thức nghiệp vụ riêng biệt theo kiểu lớp hướng đối tượng thuần túy, nhưng nó là đối tượng được thao tác xuyên suốt trong các controller như tạo sự kiện, cập nhật sự kiện, duyệt sự kiện, thống kê người tham gia và hiển thị chi tiết sự kiện. Ý nghĩa thiết kế của lớp này nằm ở chỗ nó là trung tâm của phần nghiệp vụ sự kiện: mọi thao tác từ tìm kiếm, đặt vé, thông báo cho tới kiểm soát check-in đều quy chiếu về một đối tượng sự kiện cụ thể.

Lớp Ticket

Lớp `Ticket` đại diện cho vé điện tử của người dùng đối với một sự kiện cụ thể. Các thuộc tính của lớp gồm `user`, `event`, `price`, `ticketType`, `tierName`, `seatInfo`, `qrCode`, `paymentStatus`, `checked`, `checkedAt`, `checkedBy` và `bookedAt`. Nhờ những thuộc tính này, một vé không chỉ là minh chứng cho việc mua thành công, mà còn là thực thể phản ánh trạng thái thanh toán, dữ liệu QR và trạng thái tham dự tại sự kiện.



Hình 0.6: Biểu đồ trình tự cho use case Đặt vé và xác nhận thanh toán



Hình 0.7: Biểu đồ trình tự cho use case Check-in vé bằng mã QR

Ở góc độ nghiệp vụ, lớp `Ticket` tham gia vào các thao tác như khởi tạo vé khi

người dùng đặt chỗ, cập nhật trạng thái thanh toán khi giao dịch được xác nhận, lưu dữ liệu QR để phục vụ việc check-in và cập nhật trạng thái đã sử dụng sau khi quét vé thành công. Đây là lớp có vai trò cầu nối giữa người dùng và sự kiện, bởi nó là thực thể cụ thể hóa mối quan hệ “người dùng tham dự một sự kiện nào đó thông qua một hạng vé xác định”.

Lớp Payment

Lớp Payment biểu diễn giao dịch thanh toán gắn với một hoặc nhiều vé trong hệ thống. Các thuộc tính chính gồm `user`, `event`, `tickets`, `amount`, `method`, `status` và `transactionId`. Cấu trúc này cho phép hệ thống quản lý thanh toán như một thực thể tách biệt thay vì chỉ lưu trực tiếp trạng thái tiền trên từng vé. Điều đó giúp việc truy vết giao dịch, cập nhật trạng thái thanh toán, thống kê doanh thu hoặc tích hợp cổng thanh toán thực tế trong tương lai thuận lợi hơn.

Về phương thức nghiệp vụ, lớp Payment được thao tác chủ yếu trong luồng đặt vé và xác nhận thanh toán. Khi người dùng chọn hạng vé và số lượng vé, hệ thống tạo một giao dịch ở trạng thái chờ xử lý. Sau khi xác nhận thành công, Payment được cập nhật sang trạng thái thành công, đồng thời kéo theo việc cập nhật trạng thái các vé liên quan và số lượng vé đã bán ở sự kiện tương ứng. Vì vậy, lớp này đóng vai trò như một lớp điều phối tài chính, bảo đảm sự nhất quán giữa vé, số lượng vé đã bán và trạng thái giao dịch.

Nhận xét chung về thiết kế lớp

Bốn lớp trên phản ánh lõi nghiệp vụ của hệ thống theo một cấu trúc khá mạch lạc. `User` biểu diễn tác nhân sử dụng hệ thống, `Event` biểu diễn đối tượng trung tâm mà người dùng quan tâm, `Ticket` biểu diễn quyền tham dự của người dùng đối với sự kiện, còn `Payment` biểu diễn giao dịch tài chính đi kèm với việc phát hành vé. Mối liên hệ giữa các lớp này tạo nên phần lớn luồng nghiệp vụ chính trong hệ thống. Việc thiết kế lớp theo hướng tách biệt vai trò như vậy giúp mã nguồn dễ mở rộng hơn, chẳng hạn khi cần bổ sung thêm các chức năng như hủy vé, hoàn tiền, thống kê doanh thu hay xếp chỗ theo khu vực.

0.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng MongoDB làm hệ quản trị cơ sở dữ liệu, vì vậy mô hình dữ liệu của đề tài được thiết kế theo hướng tài liệu (document-oriented) thay vì quan hệ thuần túy như trong các hệ quản trị SQL. Tuy nhiên, về bản chất nghiệp vụ, các collection trong MongoDB vẫn biểu diễn những thực thể và quan hệ tương tự một mô hình thực thể liên kết. Do đó, việc thiết kế cơ sở dữ liệu vẫn cần bắt đầu

từ việc xác định các thực thể chính, các thuộc tính quan trọng và mối liên hệ giữa chúng. Trên cơ sở đó, hệ thống được tổ chức thành các collection trung tâm như `User`, `Event`, `Ticket`, `Payment`, `Notification`, `Bookmark`, `Reminder`, `Review`, `ChatRoom`, `Message` và `Card`.

Thực thể User

Thực thể `User` lưu thông tin người dùng của hệ thống, bao gồm cả người dùng thông thường và quản trị viên. Các thuộc tính quan trọng của thực thể này gồm họ tên, email, mật khẩu đã mã hóa, avatar, quốc gia, sở thích, vị trí, mô tả hồ sơ, vai trò, trạng thái xác thực, tùy chọn nhận thông báo và token thiết bị phục vụ thông báo đẩy. Đây là thực thể trung tâm của hệ thống vì hầu hết các nghiệp vụ đều quy chiếu về người dùng, từ tạo sự kiện, đặt vé, gửi tin nhắn, nhận thông báo cho đến quản lý bookmark và lời nhắc.

Thực thể Event

Thực thể `Event` mô tả một sự kiện được tạo trong hệ thống. Các thuộc tính chính gồm tiêu đề, mô tả, danh mục, giá mặc định, ngày, thời gian, địa điểm, tọa độ, hình ảnh, số lượng người tham gia, điểm đánh giá, trạng thái sự kiện và trạng thái duyệt. Mỗi sự kiện liên kết với một người tổ chức thông qua thuộc tính `organizer`. Ngoài ra, sự kiện còn chứa một cấu trúc lồng `ticketTiers` để mô tả các hạng vé, trong đó mỗi hạng vé có tên, giá, quota và số lượng vé đã bán. Việc đặt `ticketTiers` bên trong tài liệu sự kiện là một lựa chọn phù hợp với MongoDB vì các hạng vé luôn gắn chặt với một sự kiện cụ thể và thường được truy xuất cùng nhau.

Thực thể Ticket và Payment

Thực thể `Ticket` biểu diễn vé điện tử mà người dùng nhận được khi đặt tham gia một sự kiện. Mỗi vé liên kết với một người dùng và một sự kiện, đồng thời lưu giá vé, loại vé, tên hạng vé, thông tin vị trí ghế nếu có, dữ liệu mã QR, trạng thái thanh toán, trạng thái check-in và thời điểm đặt vé. Đây là thực thể cầu nối giữa người dùng với sự kiện, phản ánh trực tiếp việc một người dùng tham gia một sự kiện nào đó qua một hạng vé xác định.

Thực thể `Payment` dùng để quản lý giao dịch thanh toán. Mỗi thanh toán liên kết với một người dùng, một sự kiện và một tập vé tương ứng. Các thuộc tính chính của nó gồm tổng số tiền, phương thức thanh toán, trạng thái giao dịch và mã giao dịch. Việc tách `Payment` thành một thực thể riêng giúp hệ thống dễ mở rộng sang

các kịch bản thực tế hơn như hoàn tiền, theo dõi giao dịch, thống kê doanh thu hoặc tích hợp cổng thanh toán thật trong tương lai.

Các thực thể hỗ trợ tương tác và theo dõi

Bên cạnh các thực thể cốt lõi, hệ thống còn có nhiều thực thể hỗ trợ cho trải nghiệm người dùng. Thực thể `Bookmark` lưu mối liên hệ giữa người dùng và sự kiện được lưu quan tâm, qua đó hỗ trợ tính năng theo dõi sự kiện. Thực thể `Reminder` lưu các lời nhắc gắn giữa người dùng và sự kiện, giúp hệ thống xác định thời điểm cần gửi nhắc lịch. Thực thể `Review` lưu đánh giá và bình luận của người dùng cho sự kiện, phục vụ tính năng phản hồi và cập nhật điểm đánh giá trung bình. Thực thể `Notification` lưu các thông báo nội bộ mà người dùng nhận được, bao gồm thông báo hệ thống, lời mời, nhắc lịch, thông báo thanh toán hoặc thông báo chat.

Ngoài ra, nhóm chức năng giao tiếp thời gian thực được tổ chức thông qua hai thực thể `ChatRoom` và `Message`. `ChatRoom` lưu thông tin một phòng chat, có thể gắn với một sự kiện và một danh sách thành viên. `Message` lưu các tin nhắn phát sinh trong từng phòng, bao gồm người gửi, nội dung, loại tin nhắn, tệp đính kèm và danh sách người đã đọc. Cuối cùng, thực thể `Card` dùng để lưu thông tin thẻ thanh toán đã lưu của người dùng ở mức tối giản, bao gồm thương hiệu thẻ, bốn số cuối, tháng hết hạn, năm hết hạn và trạng thái thẻ mặc định.

Mối quan hệ giữa các thực thể

Từ góc nhìn E-R, mối quan hệ giữa các thực thể trong hệ thống có thể mô tả như sau. Một `User` có thể tạo nhiều `Event`, trong khi mỗi `Event` chỉ có một người tổ chức chính. Một `User` có thể sở hữu nhiều `Ticket`, còn mỗi `Ticket` chỉ thuộc về một người dùng và một sự kiện. Một `Payment` có thể liên kết với nhiều `Ticket`, nhưng mỗi vé trong ngữ cảnh hệ thống hiện tại chỉ gắn với một giao dịch thanh toán tại một thời điểm. Một `User` có thể tạo nhiều `Bookmark`, `Reminder`, `Review`, `Notification`, `Card` và `Message`. Một `Event` có thể có nhiều `Ticket`, `Review`, `Notification`, `Reminder` và `Bookmark`. Đối với nhóm chat, một `ChatRoom` có thể chứa nhiều `Message` và nhiều thành viên là các `User`, còn mỗi `Message` thuộc về đúng một phòng chat và một người gửi.

Đặc điểm thiết kế dữ liệu trong MongoDB

Do sử dụng MongoDB, các quan hệ trên không được hiện thực bằng khóa ngoại như trong hệ quản trị quan hệ, mà chủ yếu thông qua các trường tham chiếu `ObjectId` và các cấu trúc lồng. Cách làm này giúp hệ thống vừa giữ được tính linh

Nhận xét chung về thiết kế cơ sở dữ liệu

```
classDiagram
    class User {
        +_id: ObjectId
        +name: String
        +email: String
        +password: String
        +avatar: String
        +country: String
        +interests: String[]
        +location: String
        +phone: String
        +description: String
        +role: String
        +verified: Boolean
        +notificationsEnabled: Boolean
        +expoPushToken: String
    }
    class Event {
        +_id: ObjectId
        +title: String
        +description: String
        +category: String
        +price: Number
        +date: Date
        +time: String
        +images: String
        +member: Number
        +rating: Number
        +status: String
        +approvalStatus: String
    }
    class Ticket {
        +_id: ObjectId
        +price: Number
        +ticketType: String
        +tierName: String
        +seatInfo: String
        +qrCode: String
        +paymentStatus: String
        +checked: Boolean
        +checkedAt: Date
        +bookedAt: Date
    }
    class Payment {
        +_id: ObjectId
        +amount: Number
        +method: String
        +status: String
        +transactionId: String
    }
    class Bookmark {
        +_id: ObjectId
    }
    class TicketTier {
        +name: String
        +price: Number
        +quota: Number
        +sold: Number
    }
    class Reminder {
        +_id: ObjectId
        +remindAt: Date
        +date sent: Boolean
    }
    class Review {
        +_id: ObjectId
        +rating: Number
        +comment: String
    }
    class Notification {
        +_id: ObjectId
        +title: String
        +message: String
        +rating: Number
        +type: String
        +isRead: Boolean
    }
    class ChatRoom {
        +_id: ObjectId
        +isGroup: Boolean
    }
    class Card {
        +_id: ObjectId
        +brand: String
        +last4: String
        +expMonth: Number
        +expYear: Number
        +isPrimary: Boolean
    }
    class Message {
        +_id: ObjectId
        +content: String
        +type: String
        +attachments: String[]
    }

    User "1" -- "0..1" Event : creates
    User "1" -- "0..1" Event : sets
    User "1" -- "0..1" Event : writes
    User "1" -- "0..1" Event : sends
    User "1" -- "0..1" Event : receives
    User "1" -- "0..1" Event : stores
    User "1" -- "0..1" Event : participates
    User "1" -- "0..1" Event : relates_to
    User "1" -- "0..1" Event : checks_in_by
    User "1" -- "0..1" Ticket : saves
    Event "1" -- "0..1" Ticket : contains
    Event "1" -- "0..1" Ticket : reminds_for
    Event "1" -- "0..1" Ticket : issues
    Ticket "1" -- "0..1" Payment : belongs_to
    Ticket "1" -- "0..1" Payment : pays_for
    Ticket "1" -- "0..1" TicketTier : owns
    Payment "1" -- "0..1" Ticket : belongs_to
    Bookmark "1" -- "0..1" Ticket : belongs_to
    TicketTier "1" -- "0..1" Reminder : makes
    Reminder "1" -- "0..1" Review : makes
    Review "1" -- "0..1" Notification : makes
    Notification "1" -- "0..1" ChatRoom : sends
    ChatRoom "1" -- "0..1" Message : contains
    Card "1" -- "0..1" Event : stores
    Message "1" -- "0..1" ChatRoom : contains
```

0.3 Xây dựng ứng dụng

Trong quá trình phát triển hệ thống, đề tài sử dụng kết hợp nhiều công cụ, ngôn ngữ lập trình, thư viện và framework ở cả phía frontend lẫn backend. Việc lựa chọn

các thành phần này dựa trên yêu cầu xây dựng ứng dụng di động đa nền tảng, backend xử lý nghiệp vụ tập trung, cơ sở dữ liệu linh hoạt và các chức năng hỗ trợ như bản đồ, thông báo đẩy, chat thời gian thực, mã QR và kiểm thử tự động. Do số lượng công cụ sử dụng tương đối nhiều, nội dung được chia thành hai bảng để thuận tiện theo dõi trong bố cục trang dọc.

Nhóm/Mục đích	Công cụ/Thư viện	Phiên bản	Vai trò sử dụng
Ngôn ngữ lập trình frontend	TypeScript	5.9.2	Phát triển ứng dụng khách với kiểu dữ liệu tĩnh, thuận tiện bảo trì và tổ chức service.
Framework mobile	Expo	54.0.33	Cung cấp nền tảng phát triển mobile-first, hỗ trợ chạy thử nhanh trên Android, iOS và web.
Framework giao diện mobile	React Native	0.81.5	Xây dựng giao diện đa nền tảng cho điện thoại và tablet.
Thư viện giao diện lõi	React	19.1.0	Quản lý component, trạng thái và vòng đời hiển thị ở frontend.
Điều hướng ứng dụng	React Navigation	7.1.18 – 7.8.5	Tổ chức điều hướng giữa các màn hình đăng nhập, sự kiện, thanh toán, chat và quản trị.
Giao diện và theme	NativeWind + Tailwind CSS + React Native Paper	4.2.1 / 3.4.18 / 5.14.5	Chuẩn hóa màu sắc, khoảng cách, kiểu component và theme cho toàn ứng dụng.
Giao tiếp API	Axios	1.13.2	Gửi yêu cầu HTTP từ frontend tới backend.
Thông báo đẩy	Expo Notifications + Expo Device	0.32.16 / 8.0.10	Gửi và nhận push notification cho thanh toán, lời mời, nhắc lịch và tin nhắn mới.
Bản đồ và vị trí	react-native-maps + expo-location + Leaflet + react-native-web	1.20.1 / 19.0.8 / 1.9.4 / 0.21.0	Hỗ trợ chọn vị trí và triển khai bản đồ tương thích cho mobile và web.
Camera và mã QR	expo-camera + react-native-qrcode-svg	17.0.10 / 6.3.21	Quét mã QR và hiển thị vé điện tử ở phía người dùng.
Hình ảnh và media	expo-image-picker	17.0.10	Chọn ảnh cho sự kiện hoặc hồ sơ người dùng.
Lưu trữ cục bộ	@react-native-async-storage/async-storage	2.2.0	Lưu token và thông tin đăng nhập ở thiết bị người dùng.
Thư viện tiện ích	dayjs + uuid	1.11.19 / 11.1.0	Định dạng thời gian và sinh định danh hỗ trợ cho các thao tác phụ trợ.

Bảng 1: Danh sách công cụ và thư viện sử dụng ở phía frontend của hệ thống

Nhóm/Mục đích	Công cụ/Thư viện	Phiên bản	Vai trò sử dụng
Ngôn ngữ lập trình backend	TypeScript	5.9.3	Xây dựng mã nguồn backend có kiểu dữ liệu tĩnh, dễ bảo trì và mở rộng.
Môi trường chạy backend	Node.js LTS	v18+	Chạy server Express, scheduler và Socket.IO cho hệ thống.
Framework web backend	Express	5.1.0	Xây dựng các route và controller cho các API nghiệp vụ.
Kết nối dữ liệu	Mongoose	8.19.1	Kết nối MongoDB, định nghĩa schema và thao tác với dữ liệu nghiệp vụ.
Hệ quản trị cơ sở dữ liệu	MongoDB	8.x ecosystem	Lưu trữ người dùng, sự kiện, vé, thanh toán, thông báo, chatroom và message theo mô hình NoSQL.
Xác thực và bảo mật	JSON Web Token + bcryptjs + helmet + cors + express-validator	9.0.2 / 3.0.2 / 7.0.0 / 2.8.5 / 7.2.1	Xác thực người dùng, mã hóa mật khẩu, bảo vệ header HTTP và kiểm tra dữ liệu đầu vào.
Thời gian thực	Socket.IO	4.8.3	Hỗ trợ chat thời gian thực và cập nhật phòng trò chuyện giữa các bên tham gia.
Email hệ thống	nodemailer	7.0.9	Gửi email khôi phục mật khẩu và các thông báo liên quan nếu cấu hình đầy đủ.
Sinh mã QR	qrcode	1.5.4	Sinh dữ liệu và ảnh mã QR cho vé điện tử ở phía backend.
Gọi dịch vụ ngoài	Axios	1.13.2	Kết nối tới các dịch vụ ngoài như AI API hoặc các endpoint tích hợp.
Công cụ phát triển backend	ts-node-dev	2.0.0	Chạy server TypeScript ở chế độ phát triển với khả năng reload khi sửa mã nguồn.
Công cụ build frontend	babel-preset-expo	54.0.10	Hỗ trợ quá trình biên dịch và đóng gói ứng dụng Expo.
Công cụ kiểm thử	Jest + ts-jest	30.2.0 / 29.4.5	Viết và chạy kiểm thử đơn vị cho controller, helper và nghiệp vụ backend.
Công cụ sinh dữ liệu mẫu	seed script	Tự xây dựng	Tạo dữ liệu thử nghiệm cho MongoDB để phục vụ kiểm thử thủ công và trình diễn.

Bảng 2: Danh sách công cụ, thư viện và môi trường phát triển ở phía backend của hệ thống

0.3.2 Kết quả đạt được

Kết quả đạt được của đề án không chỉ dừng ở mức hoàn thiện mã nguồn, mà đã hình thành một hệ thống phần mềm tương đối hoàn chỉnh cho bài toán đặt vé và quản lý sự kiện trên thiết bị di động. Sản phẩm chính của đề án gồm ba thành phần cốt lõi. Thành phần thứ nhất là ứng dụng khách được phát triển bằng React Native và Expo, chịu trách nhiệm cung cấp giao diện tương tác cho người dùng trên điện thoại và trên môi trường web phục vụ thử nghiệm. Thành phần thứ hai là máy chủ backend được xây dựng bằng Node.js, Express và TypeScript, đảm nhiệm xử lý các nghiệp vụ như xác thực người dùng, quản lý sự kiện, đặt vé, thanh toán, gửi thông báo, nhắc lịch và chat thời gian thực. Thành phần thứ ba là mô hình dữ liệu MongoDB cùng hệ thống schema Mongoose, tạo nền tảng lưu trữ và quản lý các thực thể như người dùng, sự kiện, vé, thanh toán, thông báo, bookmark, reminder, review và chat.

Ngoài ba thành phần cốt lõi trên, đề án còn tạo ra một số sản phẩm hỗ trợ quan trọng. Một là bộ dữ liệu mẫu phục vụ thử nghiệm thủ công và trình diễn hệ thống thông qua script seed cho backend. Hai là bộ kiểm thử tự động ở mức controller

cho các nhóm chức năng quan trọng như xác thực người dùng, tạo sự kiện và đặt vé. Ba là tập các tài liệu thiết kế và minh họa như biểu đồ use case, biểu đồ gói, biểu đồ trình tự, wireframe giao diện và biểu đồ E-R, giúp mô tả toàn bộ hệ thống từ góc nhìn phân tích, thiết kế đến triển khai. Nhờ đó, kết quả đạt được của đồ án không chỉ là một sản phẩm chạy được, mà còn là một bộ artefact kỹ thuật khá đầy đủ phục vụ cho việc bảo trì, mở rộng và đánh giá hệ thống.

Về mặt ý nghĩa, các sản phẩm trên cho thấy hệ thống đã đáp ứng được các luồng nghiệp vụ quan trọng nhất của bài toán. Người dùng có thể đăng nhập, tìm kiếm sự kiện, xem chi tiết, lưu sự kiện, đặt vé, nhận vé điện tử và nhận thông báo. Người tổ chức có thể tạo sự kiện, quản lý hạng vé, mời người tham gia và thực hiện check-in bằng mã QR. Quản trị viên có thể kiểm soát trạng thái duyệt sự kiện. Đồng thời, việc hệ thống đã có bản build cho backend, bản export web cho frontend và dữ liệu seed cho thấy sản phẩm không chỉ tồn tại ở mức mã nguồn mà đã có thể phục vụ trình diễn và kiểm thử trong môi trường gần với thực tế hơn.

Do dự án được tổ chức chủ yếu theo module và hàm nghiệp vụ, thay vì hoàn toàn theo cấu trúc lớp thuần túy, phần thống kê dưới đây sử dụng số lượng mô-đun, thực thể dữ liệu và thành phần chính của hệ thống để phản ánh quy mô sản phẩm một cách sát thực hơn. Bảng 3 tóm tắt các sản phẩm đầu ra chính của đồ án, còn Bảng 4 trình bày các số liệu thống kê nổi bật của ứng dụng.

Sản phẩm	Thành phần	Ý nghĩa, vai trò
Ứng dụng khách	Mã nguồn frontend React Native/Expo, các màn hình, component, service và context	Cung cấp giao diện cho người dùng cuối, hiện thực các luồng khám phá sự kiện, đặt vé, quản lý vé, chat và thông báo.
Máy chủ backend	Mã nguồn Express/-TypeScript, route, controller, middleware, service, utility	Xử lý nghiệp vụ trung tâm của hệ thống, bảo vệ API, quản lý dữ liệu và điều phối các chức năng thời gian thực.
Cơ sở dữ liệu	Các schema/model MongoDB-Mongoose cho User, Event, Ticket, Payment, Notification, Bookmark, Reminder, Review, ChatRoom, Message, Card	Lưu trữ dữ liệu nghiệp vụ và bảo đảm tính nhất quán cho các thao tác cốt lõi của hệ thống.
Bản build triển khai	backend/dist và frontend/dist	Cho phép đóng gói và triển khai sản phẩm trên môi trường chạy thử và môi trường web.
Dữ liệu thử nghiệm	Script sinh dữ liệu mẫu backend/script-s/seed.js	Hỗ trợ kiểm thử thủ công, trình diễn sản phẩm và đối chiếu dữ liệu khi phát triển.
Bộ kiểm thử	Các ca kiểm thử trong backend/tests	Hỗ trợ kiểm tra tính đúng đắn của một số controller quan trọng và giảm rủi ro hồi quy khi sửa mã nguồn.
Tài liệu thiết kế	Use case, package diagram, sequence diagram, wireframe, ERD	Hỗ trợ mô tả, đánh giá và mở rộng hệ thống ở các giai đoạn tiếp theo.

Bảng 3: Các sản phẩm đầu ra chính đạt được trong quá trình thực hiện đồ án

Chỉ số thống kê	Giá trị	Ghi chú
Tổng số dòng mã nguồn chính (không tính node_modules, dist, .expo)	20 372 dòng	Tính trên các tệp .ts, .tsx, .js của backend và frontend.
Số tệp mã nguồn phía backend (backend/src)	46 tệp	Bao gồm cấu hình, controller, middleware, model, route, service và utility.
Số tệp mã nguồn phía frontend	87 tệp	Tính trên các tệp TypeScript/TSX của frontend, không tính thư viện cài đặt.
Số màn hình nghiệp vụ frontend	35 màn hình	Tính theo số tệp trong thư mục frontend/pages.
Số component giao diện dùng lại	23 component	Tính theo số tệp trong frontend/components.
Số service phía frontend	15 service	Phục vụ gọi API, chat, thông báo, thanh toán, upload, v.v.
Số model/thực thể dữ liệu backend	11 model	Gồm User, Event, Ticket, Payment, Notification, Bookmark, Reminder, Review, ChatRoom, Message, Card.
Số controller backend	11 controller	Mỗi controller xử lý một nhóm nghiệp vụ chính của hệ thống.
Số route backend	11 route module	Tổ chức endpoint theo từng nhóm chức năng.
Số utility backend	6 utility module	Bao gồm sinh token, QR, push notification, email, scheduler, v.v.
Số tệp kiểm thử backend	4 tệp	Gồm các ca kiểm thử và helper phục vụ kiểm thử.
Dung lượng mã nguồn backend chính	256 KB	Thư mục backend/src.
Dung lượng kiểm thử và script seed	52 KB	Gồm backend/tests và backend/scripts.
Dung lượng phần giao diện frontend chính	472 KB	Thư mục frontend/pages.
Dung lượng component frontend	140 KB	Thư mục frontend/components.
Dung lượng service frontend	72 KB	Thư mục frontend/services.
Dung lượng tài nguyên frontend	272 KB	Thư mục frontend/assets, chưa tính phụ thuộc ngoài.
Dung lượng bản build backend	260 KB	Thư mục backend/dist.
Dung lượng bản export web frontend	8.2 MB	Thư mục frontend/dist, phục vụ triển khai thử nghiệm trên web.

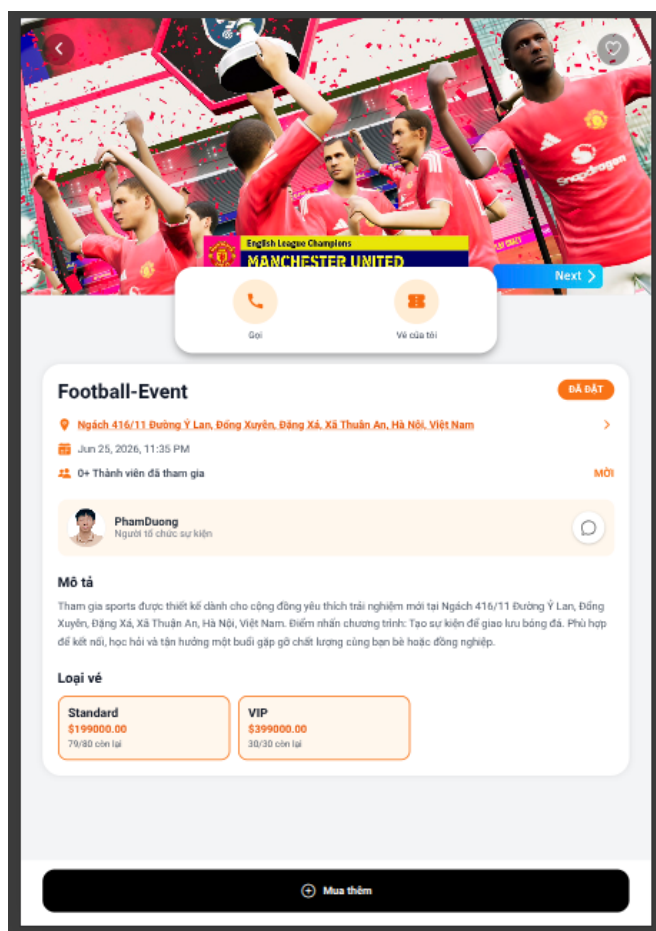
Bảng 4: Một số thông tin thống kê về quy mô và kết quả hiện thực của hệ thống

0.3.3 Minh họa các chức năng chính

Để minh họa rõ hơn kết quả hiện thực của hệ thống, đề tài lựa chọn ba chức năng tiêu biểu nhất để đưa ra hình ảnh giao diện sản phẩm, bao gồm tạo sự kiện, đặt vé và check-in vé. Đây là ba chức năng đại diện cho ba góc nhìn quan trọng của hệ thống: phía người tổ chức, phía người tham dự và phía kiểm soát việc tham gia sự kiện. Các hình minh họa dưới đây là giao diện thực tế của sản phẩm sau khi hiện thực, không phải wireframe hay giao diện khái niệm. Vì vậy, chúng phản ánh trực tiếp cách các yêu cầu nghiệp vụ đã được chuyển hóa thành các màn hình tương tác trong hệ thống.

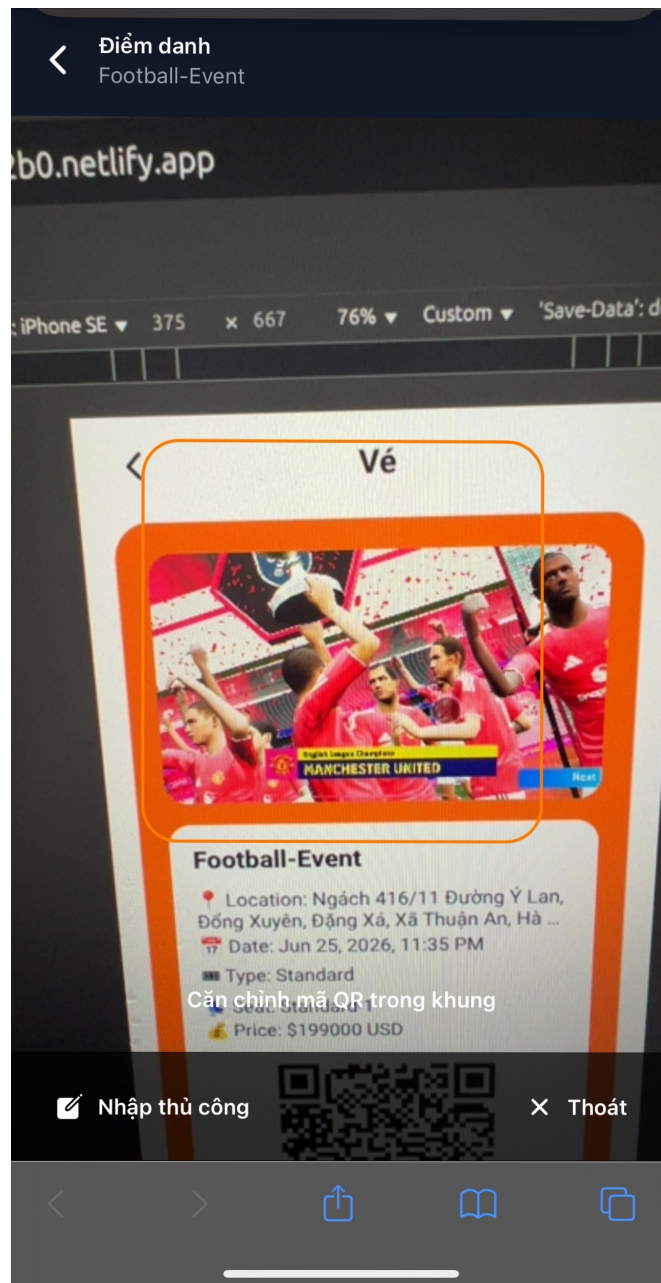
Hình 0.9: Giao diện chức năng tạo sự kiện

Hình 0.9 minh họa giao diện tạo sự kiện dành cho người tổ chức. Màn hình này cho phép nhập các thông tin chính của sự kiện như tiêu đề, mô tả, danh mục, thời gian, địa điểm và các hạng vé. Điểm đáng chú ý là giao diện được thiết kế theo luồng nhập liệu rõ ràng, giúp người tổ chức có thể tạo sự kiện từng bước mà không bị rối bởi quá nhiều thông tin trên cùng một màn hình. Ngoài ra, việc bố trí khu vực quản lý hạng vé ngay trong giao diện tạo sự kiện cho thấy hệ thống đã hỗ trợ khá tốt đặc thù nghiệp vụ của bài toán, thay vì chỉ dừng ở mức tạo một bản tin sự kiện đơn giản.



Hình 0.10: Giao diện chức năng đặt vé

Hình 0.10 minh họa giao diện đặt vé dành cho người dùng. Tại màn hình này, người dùng có thể lựa chọn hạng vé, điều chỉnh số lượng vé và xem tổng tiền trước khi xác nhận thanh toán. Giao diện tập trung làm nổi bật các yếu tố quan trọng nhất đối với quyết định mua vé, bao gồm loại vé, đơn giá, số lượng còn lại và tổng giá trị giao dịch. Cách bố trí này giúp người dùng thao tác nhanh, đồng thời hạn chế sai sót khi lựa chọn vé cho sự kiện.



Hình 0.11: Giao diện chức năng check-in vé

Hình 0.11 minh họa giao diện check-in vé bằng mã QR. Đây là một trong những chức năng thú vị nhất của hệ thống vì nó kết nối trực tiếp giữa dữ liệu vé điện tử và thao tác tham dự ngoài thực tế. Màn hình được thiết kế để người tổ chức có thể quét mã, nhận phản hồi ngay về tính hợp lệ của vé và cập nhật trạng thái tham dự

của người dùng. Chức năng này góp phần hoàn thiện vòng đời của một giao dịch đặt vé, từ lúc người dùng chọn mua vé cho tới khi được xác nhận tham dự sự kiện tại địa điểm tổ chức.

0.4 Kiểm thử

Kiểm thử là bước quan trọng để đánh giá xem hệ thống có hiện thực đúng các yêu cầu nghiệp vụ cốt lõi hay không, đồng thời giúp phát hiện sớm các lỗi xử lý dữ liệu, lỗi xác thực đầu vào và các sai lệch trong luồng nghiệp vụ giữa các thành phần của backend. Đối với hệ thống đặt vé và quản lý sự kiện trong đồ án này, không phải mọi chức năng đều có mức độ quan trọng như nhau. Những chức năng cần được ưu tiên kiểm thử trước là các chức năng trực tiếp ảnh hưởng tới trải nghiệm sử dụng và tính đúng đắn của dữ liệu, bao gồm: xác thực người dùng, tạo sự kiện, đặt vé và xác nhận thanh toán. Đây là các luồng nền tảng, bởi nếu các luồng này hoạt động sai thì hầu hết các chức năng khác như quản lý sự kiện, thông báo, nhắc lịch hay check-in cũng sẽ bị ảnh hưởng theo.

0.4.1 Mục tiêu và phạm vi kiểm thử

Trong phạm vi chương này, đề tài tập trung trình bày kiểm thử cho ba nhóm chức năng quan trọng nhất của hệ thống:

- Nhóm chức năng xác thực người dùng, bao gồm đăng ký tài khoản, đăng nhập và khởi tạo yêu cầu quên mật khẩu.
- Nhóm chức năng tạo sự kiện, bao gồm kiểm tra dữ liệu đầu vào và khởi tạo các hạng vé của sự kiện.
- Nhóm chức năng đặt vé và xác nhận thanh toán, bao gồm kiểm tra giá vé, tạo vé tạm thời, tạo giao dịch thanh toán, cập nhật trạng thái thanh toán và cập nhật số lượng vé đã bán.

Việc lựa chọn ba nhóm chức năng trên là hợp lý vì chúng phản ánh trọn vẹn chuỗi nghiệp vụ chính của hệ thống: người dùng đăng nhập vào hệ thống, người tổ chức tạo sự kiện, người tham dự chọn hạng vé và đặt vé, sau đó giao dịch được xác nhận để phát hành vé hợp lệ. Nếu ba nhóm chức năng này hoạt động ổn định thì có thể xem backend đã xử lý đúng phần nghiệp vụ cốt lõi nhất của đề tài.

Kiểm thử trong phần này tập trung vào backend, cụ thể là các controller xử lý nghiệp vụ. Đây là nơi tổng hợp nhiều điều kiện ràng buộc nhất như xác thực dữ liệu, truy cập model, gọi các hàm hỗ trợ và trả kết quả cho frontend. Ngoài ra, để hỗ trợ quá trình thử nghiệm thủ công và đối chiếu dữ liệu thực tế, đề tài còn xây dựng script sinh dữ liệu mẫu cho MongoDB, bao gồm người dùng, sự kiện, vé, thanh toán và các dữ liệu liên quan. Tuy nhiên, các ca kiểm thử tự động được trình

bày dưới đây chủ yếu là kiểm thử đơn vị, chạy độc lập và không phụ thuộc trực tiếp vào cơ sở dữ liệu thật.

0.4.2 Kỹ thuật kiểm thử và môi trường kiểm thử

Đề tài sử dụng kết hợp nhiều kỹ thuật kiểm thử nhằm vừa kiểm tra đúng đầu ra của chức năng, vừa cô lập được từng đơn vị xử lý để xác định nguyên nhân khi có lỗi. Trước hết, kỹ thuật chính được áp dụng là **kiểm thử đơn vị** (*unit testing*) ở mức controller. Mỗi ca kiểm thử tập trung vào một hành vi cụ thể của hàm xử lý, chẳng hạn khi thiếu dữ liệu đầu vào thì phải trả lỗi gì, khi dữ liệu hợp lệ thì có tạo bản ghi đúng định dạng hay không, hoặc khi giao dịch thành công thì hệ thống có cập nhật đủ các trạng thái liên quan hay không.

Về góc nhìn thiết kế ca kiểm thử, đề tài sử dụng đồng thời các kỹ thuật sau:

- **Phân lớp tương đương** (*equivalence partitioning*): chia dữ liệu đầu vào thành các nhóm hợp lệ và không hợp lệ, ví dụ email tồn tại và email không tồn tại, giá vé đúng và giá vé sai, hoặc có và không có hạng vé.
- **Kiểm thử giá trị biên và kiểm tra điều kiện ràng buộc**: tập trung vào các điều kiện dễ gây lỗi như số lượng vé nhỏ hơn 1, thiếu danh sách `ticket-Tiers`, giá thanh toán không khớp với tổng giá dự kiến hoặc trạng thái giao dịch chuyển từ `pending` sang `success`.
- **Kiểm thử hộp đen ở mức hành vi đầu ra**: xem controller như một đơn vị nhận request và trả response, từ đó kiểm tra mã trạng thái HTTP và dữ liệu JSON trả về.
- **Kiểm thử hộp trắng ở mức luồng xử lý**: dùng mock để quan sát xem controller có gọi đúng model, đúng utility và đúng số lần cần thiết hay không, ví dụ có gọi tạo `Payment`, có cập nhật `Ticket`, có phát sinh mã QR hay không.

Về công cụ, backend sử dụng Jest kết hợp `ts-jest` để chạy kiểm thử đối với mã nguồn TypeScript. Các đối tượng `req` và `res` của Express được mô phỏng bằng helper riêng để tách biệt controller khỏi môi trường chạy thật của server. Các model Mongoose như `User`, `Event`, `Ticket`, `Payment`, `Notification` và các utility như gửi email, tạo token, gửi push notification hay sinh mã QR được mock lại. Cách tiếp cận này giúp kiểm thử tập trung vào logic nghiệp vụ của controller thay vì bị phụ thuộc vào mạng, cơ sở dữ liệu hoặc dịch vụ ngoài.

Song song với kiểm thử đơn vị, đề tài chuẩn bị dữ liệu thử nghiệm bằng script seed MongoDB để hỗ trợ kiểm thử thủ công ở mức hệ thống. Dữ liệu mẫu bao gồm các tài khoản người dùng, quản trị viên, một số sự kiện với nhiều hạng vé khác nhau, vé đã đặt, đánh giá và bookmark. Dữ liệu seed giúp sinh viên có thể đăng

nhập vào ứng dụng, thử tạo sự kiện, đặt vé và quan sát phản hồi trên giao diện trong môi trường gần với vận hành thực tế hơn.

0.4.3 Thiết kế các trường hợp kiểm thử chính

Nhóm chức năng xác thực người dùng

Nhóm chức năng này được xem là điểm vào của toàn bộ hệ thống. Nếu đăng ký, đăng nhập hoặc khởi tạo quên mật khẩu xử lý sai thì người dùng sẽ không thể tiếp cận các nghiệp vụ phía sau. Bảng 5 trình bày các ca kiểm thử chính đã được thực hiện cho nhóm chức năng này.

Mã ca	Mục tiêu	Dữ liệu/điều kiện đầu vào	Kết quả mong đợi	Kết quả
AUTH-01	Đăng ký tài khoản hợp lệ	Tên, email và mật khẩu hợp lệ; email chưa tồn tại	Tạo người dùng mới, trả về mã trạng thái 201 cùng token xác thực	Đạt
AUTH-02	Từ chối đăng ký trùng email	Email đã tồn tại trong hệ thống	Trả về mã lỗi 400 và thông báo email đã được sử dụng	Đạt
AUTH-03	Đăng nhập hợp lệ	Email tồn tại, mật khẩu đúng	Trả về thông tin người dùng và token đăng nhập	Đạt
AUTH-04	Từ chối đăng nhập sai tài khoản	Email không tồn tại hoặc không tìm thấy người dùng	Trả về lỗi thông tin đăng nhập không hợp lệ	Đạt
AUTH-05	Khởi tạo quên mật khẩu	Email hợp lệ đã tồn tại trong hệ thống	Lưu mã đặt lại mật khẩu, gửi email và trả về thông báo thành công	Đạt

Bảng 5: Các trường hợp kiểm thử chính cho chức năng xác thực người dùng

Các ca kiểm thử của nhóm này chủ yếu áp dụng phân lớp tương đương và kiểm thử hộp đen. Với AUTH-01 và AUTH-03, dữ liệu đầu vào nằm trong lớp hợp lệ nên hệ thống phải trả kết quả thành công. Ngược lại, AUTH-02 và AUTH-04 đại diện cho lớp dữ liệu không hợp lệ hoặc không thỏa điều kiện nghiệp vụ, vì vậy hệ thống phải trả lỗi đúng mã trạng thái. Riêng AUTH-05 kiểm tra một luồng có thêm tác động phụ là gửi email, nên trong kiểm thử đơn vị phần gửi email được mock để xác minh controller có gọi đúng dịch vụ mà không phụ thuộc vào máy chủ thư thật.

Nhóm chức năng tạo sự kiện

Tạo sự kiện là chức năng trung tâm của phía người tổ chức. Bên cạnh các thuộc tính cơ bản như tiêu đề, danh mục, thời gian và địa điểm, hệ thống còn yêu cầu phải có ít nhất một hạng vé hợp lệ. Điều này khiến chức năng tạo sự kiện trở thành nơi thích hợp để kiểm tra các ràng buộc đầu vào và cách backend chuẩn hóa dữ liệu

trước khi lưu.

Mã ca	Mục tiêu	Dữ liệu/điều kiện đầu vào	Kết quả mong đợi	Kết quả
EVT-01	Từ chối tạo sự kiện thiếu hạng vé	Có tiêu đề, danh mục, ngày và địa điểm nhưng không có ticket-Tiers	Trả về mã lỗi 400 và thông báo cần ít nhất một hạng vé	Đạt
EVT-02	Tạo sự kiện hợp lệ	Đầy đủ thông tin sự kiện và danh sách hạng vé hợp lệ	Tạo sự kiện thành công; mỗi hạng vé được chuẩn hóa thêm trường <code>sold = 0</code>	Đạt

Bảng 6: Các trường hợp kiểm thử chính cho chức năng tạo sự kiện

Ở nhóm kiểm thử này, kỹ thuật được sử dụng nổi bật là kiểm tra điều kiện ràng buộc và kiểm thử hộp trắng ở mức luồng xử lý. Cụ thể, ca EVT-01 đại diện cho lớp dữ liệu không hợp lệ, xác minh controller phát hiện đúng trường thông tin bắt buộc còn thiếu. Ca EVT-02 kiểm tra lớp dữ liệu hợp lệ và đồng thời quan sát lời gọi tới model `Event.create` để chắc chắn hệ thống không chỉ lưu dữ liệu đầu vào thô mà còn bổ sung trạng thái mặc định như `status = upcoming`, `approvalStatus = PENDING` và số vé đã bán ban đầu bằng 0 cho từng hạng vé.

Nhóm chức năng đặt vé và xác nhận thanh toán

Đây là nhóm chức năng có nhiều bước xử lý liên tiếp nhất và ảnh hưởng trực tiếp đến tính đúng đắn của dữ liệu vé cũng như giao dịch. Khi người dùng đặt vé, hệ thống phải kiểm tra sự kiện có tồn tại không, hạng vé có hợp lệ không, số tiền có khớp với giá trị dự kiến không, sau đó mới được tạo vé tạm và tạo giao dịch thanh toán. Khi thanh toán thành công, hệ thống tiếp tục cập nhật trạng thái giao dịch, đánh dấu vé là đã thanh toán, tăng số lượng vé đã bán, cập nhật tổng số thành viên tham gia, phát sinh mã QR và tạo thông báo.

Mã ca	Mục tiêu	Dữ liệu/điều kiện đầu vào	Kết quả mong đợi	Kết quả
TKT-01	Từ chối đặt vé với giá sai	Sự kiện và hạng vé tồn tại, nhưng tổng tiền gửi lên không khớp với đơn giá nhân số lượng	Trả về lỗi 400, không tạo vé và không tạo giao dịch	Đạt
TKT-02	Đặt vé hợp lệ	Hạng vé hợp lệ, số lượng hợp lệ, tổng tiền đúng	Tạo danh sách vé trạng thái <code>pending</code> và tạo một bản ghi thanh toán trạng thái <code>pending</code>	Đạt
TKT-03	Xác nhận thanh toán thành công	Giao dịch tồn tại, thuộc đúng người dùng, tham số <code>success = true</code>	Cập nhật trạng thái thanh toán, cập nhật vé, tăng số lượng vé đã bán, tạo mã QR và thông báo	Đạt

Bảng 7: Các trường hợp kiểm thử chính cho chức năng đặt vé và xác nhận thanh toán

Nhóm này là nơi thể hiện rõ nhất việc kết hợp giữa kiểm thử hộp đen và hộp trắng. TKT-01 và TKT-02 chủ yếu kiểm tra hành vi đầu ra của controller dựa trên dữ liệu đầu vào thuộc lớp hợp lệ hoặc không hợp lệ. Trong khi đó, TKT-03 không chỉ cần kiểm tra phản hồi cuối cùng mà còn cần quan sát chuỗi thao tác nội bộ: `Payment` được cập nhật sang trạng thái thành công, `Ticket.updateMany` được gọi để đổi trạng thái vé sang `paid`, `Event.findByIdAndUpdate` được gọi để tăng số lượng vé đã bán và tiện ích sinh mã QR được gọi cho từng vé đã thanh toán. Đây là ví dụ điển hình cho kiểm thử đơn vị theo hướng cô lập phụ thuộc nhưng vẫn bám sát luồng nghiệp vụ thực tế.

0.4.4 Tổng kết kết quả kiểm thử

Sau khi xây dựng và chạy bộ kiểm thử tự động bằng `Jest`, hệ thống backend hiện có tổng cộng **3 test suite** với **10 test case** đã được thực thi thành công. Trong đó:

- 5 test case thuộc nhóm xác thực người dùng.
- 2 test case thuộc nhóm tạo sự kiện.
- 3 test case thuộc nhóm đặt vé và xác nhận thanh toán.

Kết quả chạy kiểm thử cho thấy **10/10 test case đều đạt**, tương ứng với **100% số ca kiểm thử đã thiết kế trong phạm vi hiện tại đều pass**. Điều này cho thấy các controller cốt lõi của hệ thống đang xử lý đúng các tình huống nghiệp vụ quan trọng đã được lựa chọn, đặc biệt là các kiểm tra đầu vào, luồng tạo dữ liệu và các bước cập nhật trạng thái sau giao dịch.

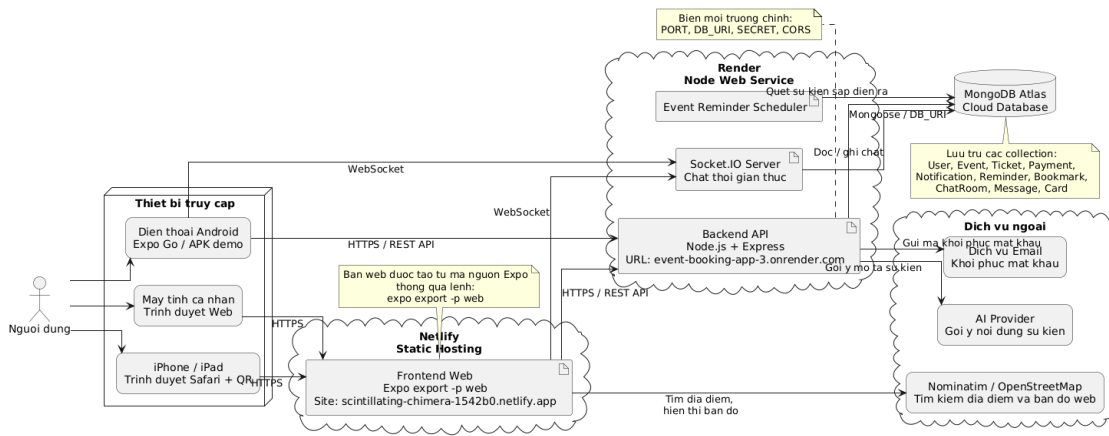
Việc toàn bộ test case đều đạt không có nghĩa là hệ thống đã hoàn toàn không còn lỗi. Thứ nhất, bộ kiểm thử hiện tại chủ yếu là kiểm thử đơn vị ở phía backend

nên chưa bao quát đầy đủ kiểm thử tích hợp giữa frontend, backend và cơ sở dữ liệu thật. Thứ hai, các luồng như chat thời gian thực, thông báo đẩy thực tế, kiểm thử hiệu năng, kiểm thử bảo mật hay kiểm thử giao diện người dùng chưa được trình bày chi tiết trong phần này. Thứ ba, một số trường hợp biên sâu hơn như tranh chấp đồng thời khi nhiều người đặt cùng một hạng vé, lỗi mạng khi gửi email hoặc lỗi đồng bộ giữa các dịch vụ ngoài cũng cần được bổ sung trong các vòng kiểm thử sau.

Trong quá trình thực hiện đồ án, nếu có trường hợp kiểm thử không đạt thì nguyên nhân thường sẽ đến từ một trong ba nhóm chính: điều kiện kiểm tra đầu vào chưa đầy đủ, luồng cập nhật dữ liệu liên quan chưa đồng bộ hoặc mock trong kiểm thử chưa phản ánh đúng hành vi thực tế của hệ thống. Tuy nhiên, ở phiên bản được trình bày trong báo cáo này, các ca kiểm thử đã được điều chỉnh lại cho phù hợp với logic hiện thực của mã nguồn và đều cho kết quả đạt yêu cầu. Các trường hợp kiểm thử phụ khác, nếu cần trình bày chi tiết hơn, có thể được đưa vào phần phụ lục để tránh làm phân tán trọng tâm của chương.

0.5 Triển khai

Trong phạm vi đồ án, hệ thống được triển khai theo mô hình client-server trên môi trường Internet, trong đó frontend đóng vai trò client và backend cung cấp các dịch vụ xử lý nghiệp vụ cũng như truy cập dữ liệu. Mô hình này phù hợp với định hướng mobile-first của hệ thống, bởi các thao tác như đăng nhập, tìm kiếm sự kiện, đặt vé, nhận thông báo, chat và check-in đều được thực hiện ở phía người dùng, sau đó gửi yêu cầu tới máy chủ backend để xử lý. Ở giai đoạn triển khai thử nghiệm, hệ thống được tổ chức thành bốn thành phần chính: frontend Expo/React Native, backend Node.js/Express, cơ sở dữ liệu MongoDB và hạ tầng triển khai trực tuyến cho phép truy cập qua Internet.



Hình 0.12: Mô hình triển khai thử nghiệm của hệ thống event-booking-app

Trong quá trình triển khai thực tế, backend được đưa lên dịch vụ Render dưới dạng một *Node Web Service*. Mã nguồn backend được đặt trong thư mục `backend`, được biên dịch từ TypeScript sang JavaScript thông qua lệnh `npm run build`, sau đó khởi động bằng lệnh `npm start`. Tập cấu hình `backend/render.yaml` được sử dụng để mô tả cấu hình triển khai, bao gồm thư mục gốc, lệnh build, lệnh start và đường dẫn health check là `/api/health`. Việc bổ sung endpoint health check cho phép kiểm tra nhanh trạng thái hoạt động của dịch vụ sau mỗi lần triển khai và hỗ trợ nền tảng Render giám sát tình trạng khởi động của server.

Backend hiện được truy cập qua địa chỉ `https://event-booking-app-3.onrender.com`. Tại phía máy chủ này, ứng dụng Node.js/Express khởi tạo từ tệp `backend/src/server.ts`, thực hiện các bước kết nối cơ sở dữ liệu, khởi động bộ lập lịch nhắc sự kiện và khởi tạo Socket.IO cho chức năng chat thời gian thực trước khi mở cổng phục vụ yêu cầu. Cổng mặc định của backend là 5000, tuy nhiên trong môi trường triển khai giá trị cổng được cấp động thông qua biến môi trường `PORT`. Các biến môi trường khác như `DB_URI`, `SECRET`, `JWT_EXPIRES_IN`, `CORS`, `EMAIL_USER`, `EMAIL_PASSWORD` hay các biến liên quan đến AI được cấu hình trong môi trường của Render thay vì ghi cứng trong mã nguồn. Cách làm này giúp hệ thống an toàn hơn và thuận tiện khi thay đổi môi trường chạy.

Cơ sở dữ liệu và kết nối dữ liệu

Ở phía dữ liệu, hệ thống sử dụng MongoDB Atlas và kết nối tới backend thông qua biến môi trường `DB_URI`. Việc đặt cơ sở dữ liệu trên một dịch vụ cloud tách biệt với backend giúp tách riêng lớp lưu trữ ra khỏi lớp xử lý nghiệp vụ, nhờ đó việc thay đổi máy chủ ứng dụng không làm ảnh hưởng trực tiếp đến dữ liệu. Với

mô hình hiện tại, dữ liệu người dùng, sự kiện, vé, thanh toán, thông báo, lời nhắc, bookmark, chatroom và message được lưu dưới dạng các collection riêng biệt. Đây là một lựa chọn phù hợp với MongoDB vì hệ thống có nhiều thực thể nghiệp vụ liên kết với nhau nhưng vẫn đòi hỏi sự linh hoạt khi mở rộng cấu trúc dữ liệu trong các giai đoạn phát triển tiếp theo.

Triển khai frontend phục vụ thử nghiệm

Ở phía client, hệ thống ban đầu được xây dựng như một ứng dụng di động bằng React Native và Expo. Tuy nhiên, trong quá trình thử nghiệm thực tế, việc phân phối ứng dụng lên iOS gặp ràng buộc về tài khoản Apple Developer. Vì vậy, để thuận tiện cho việc trình diễn và kiểm thử trên nhiều thiết bị, đồ án sử dụng thêm một phương án triển khai web từ chính mã nguồn Expo. Cụ thể, frontend được export ra dạng website tĩnh bằng lệnh `expo export -p web`, sinh ra thư mục `frontend/dist`. Thư mục này sau đó được triển khai lên Netlify theo hình thức static hosting và truy cập qua tên miền thử nghiệm `https://scintillating-chimera-1542b0.netlify.app`. Nhờ đó, người dùng có thể mở hệ thống trên iPhone hoặc máy tính bằng trình duyệt thông qua đường link hoặc mã QR mà không cần cài trực tiếp ứng dụng native.

Trong quá trình chuyển frontend sang môi trường web, một số màn hình phụ thuộc thành phần native đã được điều chỉnh để tương thích. Điển hình là màn hình chọn vị trí `SelectLocation`: trên mobile, màn hình này dùng `react-native-maps`, còn trên web hệ thống sử dụng tệp riêng `SelectLocation.web.tsx` với thư viện Leaflet và bản đồ OpenStreetMap để vẫn bảo đảm người dùng có thể tìm kiếm địa chỉ, xem bản đồ và chọn vị trí tương tự như trên điện thoại. Điều này cho thấy hệ thống không chỉ được triển khai về mặt hạ tầng mà còn được thích nghi ở mức giao diện để phù hợp với môi trường vận hành thực tế.

Liên kết giữa frontend và backend

Sau khi backend đã được triển khai trên Render, frontend được cấu hình sử dụng địa chỉ API công khai `https://event-booking-app-3.onrender.com` trong lớp service giao tiếp. Ở phía backend, biến môi trường CORS được đặt theo tên miền frontend trên Netlify để chỉ cho phép client hợp lệ gửi yêu cầu đến server. Nhờ đó, quá trình đăng nhập, lấy danh sách sự kiện, đọc dữ liệu người dùng và các luồng nghiệp vụ chính có thể được kiểm thử trong môi trường thực tế qua Internet, thay vì chỉ chạy trong mạng nội bộ. Việc kiểm tra backend được thực hiện thông qua endpoint `/api/health`, còn frontend được kiểm tra bằng cách export lại website sau mỗi lần thay đổi cấu hình và triển khai lại thư mục `dist` lên Netlify.

Thiết bị và cấu hình thử nghiệm

Về phía thiết bị người dùng, hệ thống được thử nghiệm trên điện thoại thông minh Android, điện thoại iPhone và trình duyệt web trên máy tính cá nhân có kết nối Internet ổn định. Với Android, ứng dụng có thể chạy qua Expo Go hoặc tiếp tục đóng gói thành bản APK để cài đặt thủ công. Với iOS, phương án trình diễn chính trong đề án là truy cập bản web triển khai trên Netlify bằng Safari thông qua đường link hoặc mã QR. Về mặt máy chủ, hệ thống không sử dụng một máy vật lý cài đặt thủ công mà tận dụng mô hình *Platform as a Service*: backend chạy trên Render, frontend tĩnh chạy trên Netlify và dữ liệu được lưu trên MongoDB Atlas. Đây là cách triển khai phù hợp với quy mô đề án vì đơn giản, ít tốn công quản trị hạ tầng nhưng vẫn đủ để tái hiện quá trình vận hành của một hệ thống thực tế có nhiều thành phần phân tán.

Đánh giá mô hình triển khai

Ở mức thử nghiệm, mô hình triển khai hiện tại đã đủ để kiểm tra các luồng nghiệp vụ chính của hệ thống trong môi trường thực tế có kết nối mạng, bao gồm đăng nhập, đọc dữ liệu từ server, tương tác với bản đồ web, quản lý sự kiện và trao đổi dữ liệu qua API. Mô hình này cũng cho phép nhóm phát triển dễ dàng cập nhật backend và frontend một cách độc lập. Tuy nhiên, nếu triển khai ở quy mô lớn hơn trong tương lai, hệ thống vẫn cần được bổ sung thêm các cơ chế như giám sát tài nguyên, lưu log tập trung, tối ưu hiệu năng truy vấn, cấu hình database theo chế độ tin cậy cao, triển khai CDN cho tài nguyên tĩnh và tăng cường bảo mật đối với khóa bí mật, email, token và dữ liệu người dùng.